Chapter 04: Testing

Testing is an important phase for validation, correctness, reliability, performance, and robustness of the Smart Waste Detection and Segregation Platform.

4.1 Testing Strategy

We applied a mix of testing strategies like Unit Testing, Integration Testing, Model Testing, and User Interface Testing.

4.1.1 Unit Testing

Performed on:

- Image Pre-processing functions
- Prediction pipeline
- FastAPI endpoints
- React components that handle file uploads and camera inputs

Tools Used:

- Python (unittest)
- Postman / ThunderClient
- Browser DevTools
- TensorFlow internal model testing utilities

4.1.2 Integration Testing

- ReactJS Frontend → FastAPI Backend
- Backend → MobileNetV2 Model
- Camera Classifier → Backend API

Tools Used:

- Axios API calls (frontend)
- FastAPI Swagger UI (http://127.0.0.1:8000/docs)
- Manual integration checks

4.1.3 Model Testing

The MobileNetV2 model was evaluated on:

- Accuracy
- Confusion matrix
- Classification report
- Prediction latency

Tools Used:

- TensorFlow
- NumPy
- Scikit-learn

- Matplotlib (optional)

Results:

- 94% overall accuracy
- Excellent balance between accuracy and inference speed

4.1.4 Functional Testing

We ensured all platform features work properly and verified each:

Table 4.1 - Functional Test Results

Feature	**Status**
Image upload	Working
Waste classification	Working
Confidence score	Working
Camera-based detection	Working
Waste management suggestions	Working
API response handling	Working
UI responsiveness	Working

4.1.5 User Interface Testing

Tested the usability and clarity of:

- Upload button
- Preview area
- Camera modal
- Result card styling

Tools:

- Browser manual testing
- Lighthouse UI testing
- Mobile responsiveness checks

4.1.6 Error Handling Tests

Tested scenarios:

- Uploading non-image file
- No file selected
- Corrupted image
- Backend down
- Slow network

Appropriate error messages were displayed.

4.2 Test Cases and Outcomes

Below is a structured table of test cases executed during the testing phase.

4.2.1 Functional Test Cases

Table 4.2 – API & Model Functional Tests

Test Case ID	Description	Input	Expected Output	Actual Output	Result
TC01	Check if backend loads	GET /	JSON message returned	Same	PASS
TC02	Upload valid waste image	JPG image	Predicted class + confidence	Same	PASS
TC03	Upload corrupted image	Broken file	Error message	Same	PASS
TC04	Upload non-image file	PDF/TXT	"Invalid image format"	Correct message	PASS
TC05	Model prediction speed	Single image	Prediction < 1.5 sec	~1.0 sec	PASS

TC06	Prediction correctness	50 test images	>90% match	94% accuracy	PASS

4.2.2 Camera Detection Tests

Table 4.3 – Real-Time Camera Test Cases

Test Case ID	**Description**	**Expected Output**	**Actual Output**	**Result**
TC07	Start camera stream	Live preview visible	Working	PASS
TC08	Capture frame & send	Prediction result	Working	PASS
TC09	Low-light image	Acceptable confidence	Slight drop but acceptable	PASS
TC10	Moving objects	Smooth frame capture	Working	PASS

4.2.3 Frontend UI Tests

Table 4.4 - ReactIs UI Tests

Test Case ID	Test	Expected Output	Actual Output	Result
TC11	Upload preview	Show image thumbnail	Works perfectly	PASS
TC12	No file selected	Alert message	Same	PASS
TC13	Display waste info	Show carbon, recycle, disposal tips	Works	PASS
TC14	Button disabled while loading	Button shows "Analyzing…"	Working	PASS
TC15	Mobile view UI	Fully responsive	Good UI	PASS

4.2.4 Model Evaluation Test Cases

Table 4.5 – ML Testing Results

Metric	**Expected**	**Actual**	**Result**
Accuracy	≥ 90%	**94%**	PASS
Precision	High	0.93	PASS
Recall	High	0.93	PASS
F1-Score	High	0.93	PASS
Confusion Matrix	Minimal misclassification	Acceptable	PASS

Summary of Testing Results

- Every essential feature operated as planned.
- Strong generalization and high accuracy (94%) were attained by the model.
- Both camera-based detection and image upload were dependable.
- The user interface is clear, responsive, and successfully conveys waste management insights.
- Every API endpoint responded appropriately to both valid and incorrect data.